Proyecto de Diseño 1

Interruptores, Luces, y Multiplexores

El objetivo de esta práctica es aprender como conectar entradas y salidas a un circuito implementado en un dispositivo reconfigurable. Se empleará la tarjeta de desarrollo DE2 para implementar dicho circuito, usando los interruptores SW_{17-0} como entradas y los diodos emisores de luz (LEDs por sus siglas en inglés), así como los visualizadores de siete segmentos, como salidas.

Parte 1

La tarjeta de desarrollo DE2 proveé 18 interruptores conmutadores, llamados SW_{17-0}, que pueden usarse como entradas a un circuito. También contiene 18 diodos emisores de luz roja, llamados $LEDR_{17-0}$, que pueden usarse para visualizar los valores de salida de un circuito. El códio mostrado en la Figura 1 muestra una entidad VHDL sencilla que usa los interruptores y muestra su estado en los LEDs rojos. Dado que hay 18 interruptores y LEDs, es conveniente representarlos como arreglos. En el código mostrado se hace una sola asignación para las 18 salidas, que es equivalente a las 18 asignaciones individuales `LEDR(17)<=SW(17); LEDR(16)<=SW(16); ... LEDR(0)<=SW(0);`

```
1  - - Circuito sencillo que conecta los interrupt ores SW a los LEDs rojos
2  ENTITY parte1 IS
3          PORT (  SW : IN BIT_VECTOR(17 DOWNTO 0);
4                  LEDR : OUT BIT_VECTOR(17 DOWNTO 0)); - - LEDs rojos
5  END parte1;
6
7  ARCHITECTURE Behavior OF part1 IS
8  BEGIN
9          LEDR <= SW;
10 END Behavior
```

Figura 1: Entidad sencilla que muestra el estado de los interruptores en los LEDs rojos.

Los interruptores y LEDs en la tarjeta de desarrollo DE2 están directamente conectados a patillas específicas del dispositivo reconfigurable. Para usarlos en sus diseños, es necesario especificar la asignación correcta de patillas, la cual está descrita en el *Manual de Usuario de la tarjeta DE2*. Por ejemplo, el manual especifica que el interruptor SW_0 está conectado a la patilla $N25$ del dispositivo reconfigurable, y el diodo emisor de luz $LEDR_0$ está conectado a la patilla $AE23$. La asignación de cada patilla del dispositivo reconfigurable se encuentra almacenada en el archivo *DE2_pin_assignments.csv*, el cual se encuentra dentro del disco compacto *DE2 System CD* proporcionado junto con la tarjeta de desarrollo; también se puede localizar en el sitio web del Programa Universitario de Altera. Para poder usar cualquier elemento de la tarjeta de desarrollo en sus diseños es necesario que el archivo *DE2_pin_assignments.csv* sea importado al entorno integrado de desarrollo (IDE por sus siglas en inglés) Quartus II. El procedimiento de importación se indica en el tutorial *Quartus II Introduction using VHDL Design*, también disponible através del sitio web del Programa Universitario de Altera.

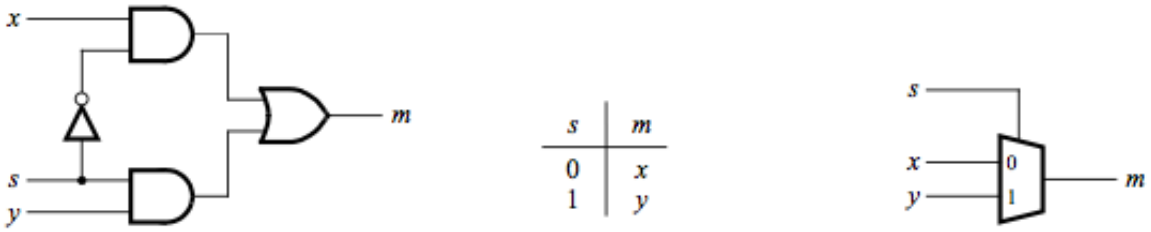
Es importante resaltar que la asignación de patillas en el archivo *DE2_pin_assignments.csv* será útil solo si los nombres asignados a cada patilla coinciden con los nombres de los puertos de entrada/salida que se usan en sus diseños. Por ejemplo, el archivo de asignación usa los nombres $SW[0] \ldots SW[17]$ y $LEDR[0] \ldots LEDR[17]$ para los interruptores y diodos emisores de luz, por lo cual estos mismos nombres son usados en el código mostrado en la Figura 1 (note que el IDE Quartus II usa corchetes cuadrados para denotar arreglos, mientras que la sintaxis correspondiente en VHDL es con paréntesis).

Ejecute los siguientes pasos para implementar un circuito correspondiente al códio mostrado en la Figura 1.

1. Cree un nuevo proyecto en Quartus II. Seleccione el dispositivo Cyclone II EP2C35F672C6 como destino de su diseño, que es el dispositivo reconfigurable usado en la tarjeta DE2.
2. Cree una nueva entidad VHDL conforme al código mostrado en la Figura 1 e incluyala en su proyecto.
3. Incluya en su proyecto la asignación de patillas para la tarjeta DE2 como se planteó anteriormente. Compile su proyecto.
4. Programe el dispositivo reconfigurable con su diseño. Pruebe la funcionalidad del circuito cambiando el estado de los interruptores y observando el cambio en los LEDs.

Parte 2

La Figura 2a muestra un circuito que implementa un multiplexor 2 a 1 con entrada de selección s. Si $s = 0$, la salida del multiplexor m es igual a la entrada x, y si $s = 1$, la salida es igual a y. La tabla de verdad del multiplexor y su correspondiente símbolo son mostrados en las Figuras 2b y 2c.



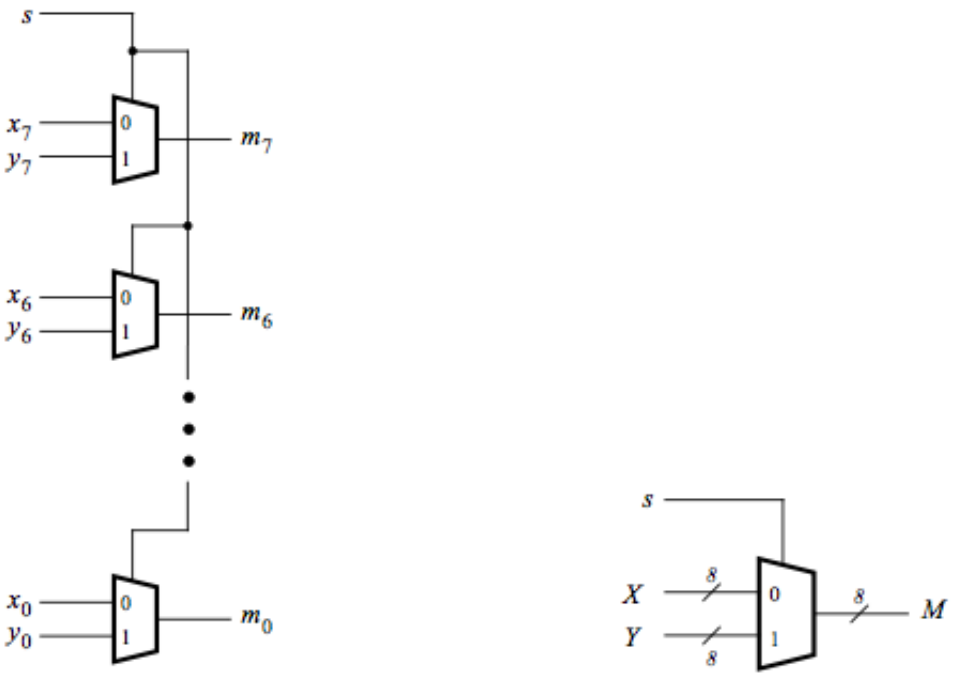
(a) Circuito (b) Tabla de verdad (c) Símbolo

Figura 2: Multiplexor 2 a 1.

El multiplexor mostrado en la Figura 2 es descrito mediante la sentencia en VHDL

```
m<=(NOT (s) AND x) OR (s AND y);
```

El circuito mostrado en la Figura 3a se conoce como multiplexor 2 a 1 de ocho bits de ancho. Dicho circuito contiene dos entradas de ocho bits cada una, nombradas X y Y, y produce una salida de ocho bits M, como se muestra en la Figura 3b, donde las entradas y salidas son representadas como conjuntos de datos. Si $s = 0$, entonces $M = X$, mientras que si $s = 1$, entonces $M = Y$. El multiplexor 2 a 1 de ocho bits de ancho, puede ser descrito mediante ocho asignaciones como la mostrada en el párrafo anterior.



(a) Circuito (b) Símbolo

Figura 3: Multiplexor 2 a 1 de ocho bits de ancho.

Implemente el circuito mostrado en la Figura 3 como una entidad VHDL mediante los siguientes pasos:

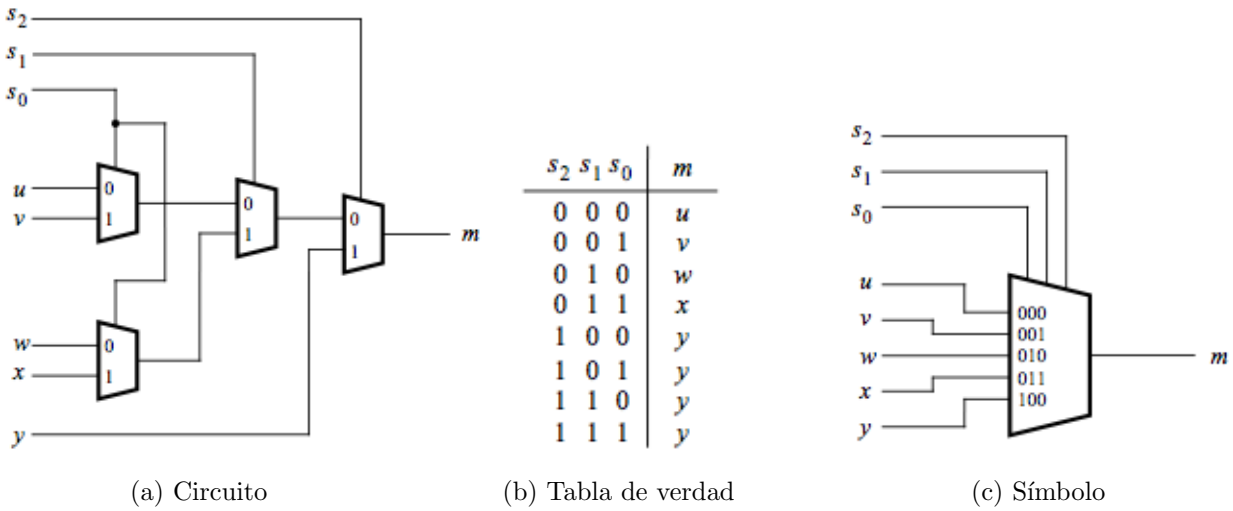
1. Cree un nuevo proyecto en el IDE Quartus II, seleccionando como destino de su diseño el dispositivo reconfigurable EP2C35F672C6 de la familia Cyclone II.
2. Incluya en su proyecto el archivo con el cádigo VHDL que implementa el multiplexor 2 a 1 de ocho bits de ancho. Use el interruptor SW_{17} en la tarjeta de desarrollo DE2 como la entrada de selección, los interruptores SW_{7-0} como la entrada X, y los interruptores SW_{15-8} como la entrada Y. Conecte los diodos emisores de luz $LEDR$ a los interruptores para monitorear su estado, y la salida M del multiplexor a los diodos emisores de luz $LEDG_{7-0}$.
3. Compile su proyecto.
4. Programe el dispositivo reconfigurable con su diseño. Pruebe la funcionalidad del multiplexor 2 a 1 conmutando el estado de los interruptores y observando el estado de las salidas en $LEDR$.

Parte 3

En la Figura 2 se muestra un multiplexor 2 a 1 que selecciona entre dos entradas x y y. Para esta parte, considere un circuito que seleccione la salida m de entre cinco posibles entradas u, v, w, x y y. La Figura 4a muestra como se puede construir el multiplexor 5 a 1 usando cuatro multiplexores 2 a 1. Este circuito emplea una entrada de selección de tres bits $s_2s_1s_0$, e implementa la tabla de verdad mostrada en la Figura 4b. El símbolo para el multiplexor 5 a 1 se muestra en la Figura 4c.

Recuerde que el multiplexor 2 a 1 de ocho bits de ancho, mostrado en la Figura 3, puede ser implementado usando ocho instancias de un multiplexor 2 a 1. En la Figura 5 se aplica el concepto anterior para

definir un multiplexor 5 a 1 de tres bits de ancho, el cual contiene tres instancias del circuito mostrado en la Figura 4a.



s_2	s_1	s_0	m
0	0	0	u
0	0	1	v
0	1	0	w
0	1	1	x
1	0	0	y
1	0	1	y
1	1	0	y
1	1	1	y

(a) Circuito (b) Tabla de verdad (c) Símbolo

Figura 4: Multiplexor 5 a 1.

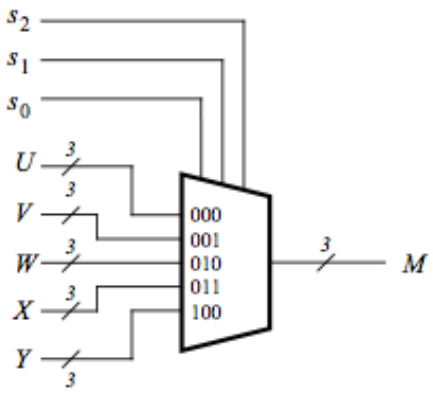


Figura 5: Multiplexor 5 a 1 de tres bits de ancho.

Implemente el multiplexor 5 a 1 mostrado en la Figura 4 en la tarjeta de desarrollo DE2 siguiendo los siguientes pasos.

1. Cree un nuevo proyecto en el IDE Quartus II, seleccionando como destino de su diseño el dispositivo reconfigurable EP2C35F672C6 de la familia Cyclone II.
2. Añada a su proyecto un nuevo archivo de diseño que contenga el código VHDL que describa su circuito.
3. Use los interruptores SW_{17-15} como las entradas de selección, y divida los interruptores remanentes para implementar cada una de las entradas U a Y. Conecte los interruptores a los diodos emisores de luz $LEDR$ para monitorear su estado, y la salida del multiplexor a los diodos emisores de luz $LEDG_{2-0}$.
4. Incluya la asignación correcta de patillas a su diseño y compilelo.

5. Programe el dispositivo reconfigurable con tu diseño. Pruebe la funcionalidad del multiplexor 5 a 1 de tres bits de ancho cambiando el estado de los interruptores y observando que la salida obedece a la tabla de verdad mostrada en la Figura 4b.

Parte 4

La Figura 6 muestra un módulo decodificador de siete segmentos con una entrada de tres bits $c_2c_1c_0$. Este decodificador produce siete salidas que son usadas para desplegar un caracter en un visualizador de siete segmentos. El caracter a desplegar es determinado por el valor de las entradas. La Tabla 1 muestra los caracteres y sus correspondientes códigos. Para mantener un diseño sencillo, solo cuatro caracteres son incluidos en la Tabla 1; adicionalmente el caracter que representa el espacio en blanco fue asignado a los códigos 100-111.

Los siete segmentos en el visualizador son identificados con siete índices (números del 0 al 6), mostrados en la Figura 6. Cada segmento se ilumina cuando se le asigna un cero lógico a su correspondiente entrada. El objetivo de esta parte es escribir un programa en VHDL que contenga las funciones lógicas necesarias para implementar el decodificador de siete segmentos descrito por la Tabla 1. Use solo expresiones booleanas para definir las salidas.

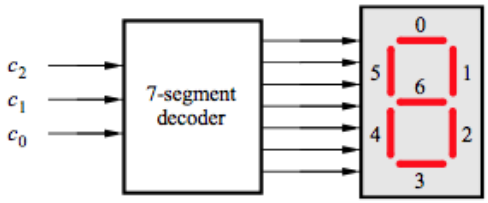


Figura 6: Decodificador de siete segmentos.

$c_2c_1c_0$	Caracter
000	'S'
001	'I'
010	'E'
011	't'
100 - 111	' '

Tabla 1: Códigos para cada caracter

Ejecute los siguientes pasos para implementar su diseño:

1. Cree un nuevo proyecto en Quartus II.
2. Añada a su proyecto un archivo de diseño nuevo que contenga el código VHDL que describa su circuito. Asigne las entradas $c_2c_1c_0$ a los interruptores SW_{2-0}, y las salidas del decodificador al visualizador $HEX0$ de la tarjeta de desarrollo DE2. Para poder usar el visualizador de siete segmentos $HEX0$ en la tarjeta de desarrollo, el puerto de salida de su diseño debe de ser declarado como

```
HEX0: OUT BIT_VECTOR(0 to 6);
```

3. Los nombres de cada segmento en el visualizador son $HEX0_0$, $HEX0_1$, ..., $HEX0_6$

4. Después de haber realizado la correcta asignación de patillas, compile su proyecto.
5. Programe su diseño en el dispositivo reconfigurable y pruebe su funcionamiento.

Parte 5

El circuito mostrado en la Figura 7 usa un multiplexor 5 a 1 de tres bits de ancho para habilitar la selección de uno de cinco caracteres. El caracter seleccionado se mostrará en un visualizador de siete segmentos usando el decodificador diseñado en la Parte 4. Cada código mostrado en la Tabla 1 es asignado a una de las entradas del multiplexor SW_{14-0}, el caracter a desplegar se establece mendiante las entradas de selección en el multiplexor SW_{17-15}.

En la Figura 8 se muestra un bosquejo del código en VHDL que describe el circuito de la Figura 7; note que en este código se han usado los diseños de la Parte 3 y 4 como subcircuitos. El objetivo de esta sección es expandir el diseño mostrado en la Figura 7 de forma que se utilicen cinco visualizadores de siete segmentos para desplegar la palabra "SIEtE", y rotarla en forma circular al cambiar las entradas de selección SW_{17-15} en el multiplexor. El patrón de rotación es mostrado en la Tabla 2.

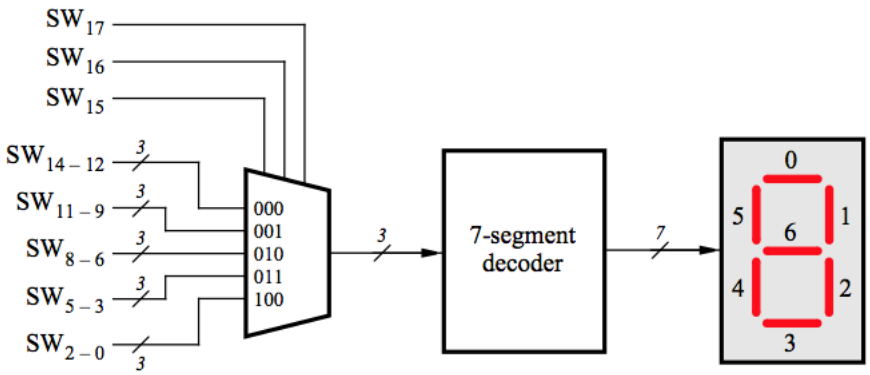


Figura 7: Decodificador de siete segmentos.

$SW_{17}SW_{16}SW_{15}$	Cadena a desplegar				
	HEX4	HEX3	HEX2	HEX1	HEX0
000	'S'	'I'	'E'	't'	'E'
001	'I'	'E'	't'	'E'	'S'
010	'E'	't'	'E'	'S'	'I'
011	't'	'E'	'S'	'I'	'E'
100	'E'	'S'	'I'	'E'	't'

Tabla 2: Patrón de rotación de la cadena 'SIEtE'

Realice los siguientes pasos para implementar su diseño:

1. Cree un nuevo proyecto en el IDE Quartus II, seleccionando como destino de su diseño el dispositivo reconfigurable EP2C35F672C6 de la familia Cyclone II.
2. Incluya el códio VHDL que describa su diseño. Asigne los interruptores SW_{17-15} de la tarjeta DE2 como las entradas de selección de cada una de las instancias de el multiplexor 5 a 1 de tres bits de

ancho. También asigne, en el orden adecuado, los interruptores SW_{14-0} como entradas a los multiplexores, de forma que el patrón de rotación en la Tabla 2 aparezca en los visualizadores de forma correcta. Finalmente, asigne los visualizadores de siete segmentos $HEX4$, $HEX3$, $HEX2$, $HEX1$ y $HEX0$ como salidas de los multiplexores.

3. Incluya el archivo de asignación de patillas adecuado para la tarjeta DE2.
4. Programe su diseño en el dispositivo reconfigurable. Pruebe su funcionamiento, confirmando que el patrón se despliega siguiendo el patrón indicado en la Tabla 2.

Parte 6

Expanda su diseño anterior de forma que use los ocho visualizadores de siete segmentos en la tarjeta DE2. Su circuito deberá ser capaz de desplegar palabras de cinco (o menos) caracteres en los ocho visualizadores, y también de rotar la palabra desplegada cuando los interruptores SW_{17-15} sean conmutados siguiendo el patrón mostrado en la Tabla 3.

$SW_{17}SW_{16}SW_{15}$	Cadena a desplegar							
	HEX7	HEX6	HEX5	HEX4	HEX3	HEX2	HEX1	HEX0
000	' '	' '	' '	'S'	'I'	'E'	't'	'E'
001	' '	' '	'S'	'I'	'E'	't'	'E'	' '
010	' '	'S'	'I'	'E'	't'	'E'	' '	' '
011	'S'	'I'	'E'	't'	'E'	' '	' '	' '
100	'I'	'E'	't'	'E'	' '	' '	' '	'S'
101	'E'	't'	'E'	' '	' '	' '	'S'	'I'
110	't'	'E'	' '	' '	' '	'S'	'I'	'E'
111	' E'	' '	' '	' '	'S'	'I'	'E'	't'

Tabla 3: Patrón de rotación de la cadena 'SIEtE'

Realice los siguientes pasos para implementar su diseño:

1. Cree un nuevo proyecto en el IDE Quartus II, seleccionando como destino de su diseño el dispositivo reconfigurable EP2C35F672C6 de la familia Cyclone II.
2. Incluya la descipción de su diseño en código VHDL. Asigne los interruptores SW_{17-15} a las entradas de selección de cada uno de los multiplexores instancias en su diseño. También conecte los interruptores SW_{14-0} a cada instancia de los multiplexores como se necesite para producir el patrón de caracteres mostrado en la Tabla 3. Conecte las salidas de los multiplexores a los visualizadores de siete segmentos $HEX7, \ldots, HEX0$.
3. Incluya la asignación de pines adecuada para la tarjeta DE2 para los interruptores, LEDs, y visualizadores de siete segmentos. Compile su proyecto.
4. Programe el dispositivo reconfigurable y pruebe la funcionalidad de su diseño en la tarjeta DE2.

```
1  ENTITY parte5 IS
2          PORT (  SW      : IN    BIT_VECTOR(17 DOWNTO 0);
3                  HEX0    : OUT   BIT_VECTOR(0 TO 6));
4  END parte5;
5
6  ARCHITECTURE Behavior OF parte5 IS
7          COMPONENT mux_3bit_5to1
8                  PORT (  S, U, V, W, X, Y        : IN    BIT_VECTOR(2 DOWNTO 0);
9                          M                       : OUT   BIT_VECTOR(2 DOWNTO 0));
10         END COMPONENT;
11
12         COMPONENT char_7seg
13                 PORT (  C       : IN    BIT_VECTOR(2 DOWNTO 0);
14                         Display : OUT   BIT_VECTOR(0 TO 6));
15         END COMPONENT;
16
17         SIGNAL M : BIT_VECTOR(2 DOWNTO 0);
18
19 BEGIN
20         M0: mux_3bit_5to1 PORT MAP (SW(17 DOWNTO 15), SW(14 DOWNTO 12), SW(11 DOWNTO 9),
21                                     SW(8 DOWNTO 6), SW(5 DOWNTO 3), SW(2 DOWNTO 0), M);
22         H0: char_7seg PORT MAP (M, HEX0);
23 END Behavior;
24
25 - - Implementa un multiplexor 5 a 1 de 3 bits de ancho
26 ENTITY mux_3bit_5to1 IS
27         PORT (  S, U, V, W, X, Y : IN  BIT_VECTOR(2 DOWNTO 0);
28                 M                : OUT BIT_VECTOR(2 DOWNTO 0));
29 END mux_3bit_5to1;
30
31 ARCHITECTURE Behavior OF mux_3bit_5to1 IS
32
33         - - inserte su código aqui
34
35 END Behavior;
36
37 ENTITY char_7seg IS
38         PORT (  C       : IN    BIT_VECTOR(2 DOWNTO 0);
39                 Display : OUT   BIT_VECTOR(0 TO 6));
40 END char_7seg;
41
42 ARCHITECTURE Behavior OF char_7seg IS
43
44         - - inserte su código aqui
45
46 END Behavior;
```

Figura 8: Descripción en VHDL del circuito mostrado en la Figura 7.